



FILE D'ATTENTE (QUEUE)

Introduction

Lors de la création de votre application Web, il se peut que certaines tâches, telles que l'analyse et le stockage d'un fichier CSV téléchargé, prennent trop de temps à exécuter lors d'une requête Web classique. Heureusement, Laravel vous permet de créer facilement des tâches en file d'attente qui peuvent être traitées en arrière-plan. En déplaçant les tâches à forte intensité de temps vers une file d'attente, votre application peut répondre aux demandes Web à une vitesse fulgurante et offrir une meilleure expérience utilisateur à vos clients.

Introduction

Les options de configuration de la file d'attente de Laravel sont stockées dans le fichier de configuration **config/queue.php** de votre application. Dans ce fichier, vous trouverez des configurations de connexion pour chacun des pilotes de file d'attente inclus dans le framework, y compris les pilotes de base de données, **Amazon SQS**, **Redis** et **Beanstalkd**, ainsi qu'un pilote synchrone qui exécutera les tâches immédiatement (à utiliser pendant le développement local). Un pilote de file d'attente **null** est également inclus, qui rejette les travaux en file d'attente.

Le pilote Database

Afin d'utiliser le pilote de file d'attente de base de données, vous aurez besoin d'une table de base de données pour contenir les travaux. Pour générer une migration qui crée cette table, exécutez la commande Artisan **queue:table**. Une fois la migration créée, vous pouvez migrer votre base de données à l'aide de la commande **migrate** :

```
php artisan queue:table  
php artisan migrate
```

Enfin, n'oubliez pas de demander à votre application d'utiliser le pilote de **database** en mettant à jour la variable **QUEUE_CONNECTION** dans le fichier `.env` de votre application.



CRÉATION D'UN JOB

Génération de classes job

Par défaut, tous les travaux pouvant être mis en file d'attente pour votre application sont stockés dans le répertoire **app/Jobs**. Si le répertoire **app/Jobs** n'existe pas, il sera créé lorsque vous exécuterez la commande Artisan **make:job** :

```
php artisan make:job ProcessPodcast
```

La classe générée mettra en œuvre l'interface **Illuminate\Contracts\Queue\ShouldQueue**, indiquant à Laravel que le travail doit être poussé dans la file d'attente pour être exécuté de manière **asynchrone**.

Structure de la classe job

Les classes de travail sont très simples et ne contiennent normalement qu'une méthode de gestion qui est invoquée lorsque le travail est traité par la file d'attente.

```
class ProcessPodcast implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;
    public $podcast;
    public function __construct(Podcast $podcast)
    {
        $this->podcast = $podcast;
    }
    public function handle(AudioProcessor $processor)
    {
        // Process uploaded podcast...
    }
}
```



JOB MIDDLEWARE

Job Middleware

Les job middlewares vous permettent d'envelopper une logique personnalisée autour de l'exécution des tâches mises en file d'attente, réduisant ainsi la quantité de texte passe-partout dans les tâches elles-mêmes.

Laravel n'a pas d'emplacement par défaut pour le job middleware, vous pouvez donc placer le job middleware n'importe où dans votre application. Dans cet exemple, nous placerons le middleware dans un répertoire **app/Jobs/Middleware**

Job Middleware

```
<?php
namespace App\Jobs\Middleware;

class JobMiddlewareExample
{
    public function handle($job, $next)
    {
        // Effectuer les taches ici
    }
}
```

Job Middleware

Après avoir créé un job middleware, il peut être attaché à un job en le renvoyant depuis la méthode **middleware** du job. Cette méthode n'existe pas sur les jobs créés par la commande **Artisan make:job**, vous devrez donc l'ajouter manuellement à votre classe de job :

```
public function middleware()  
{  
    return [new JobMiddlewareExample];  
}
```



DISTRIBUER LES JOBS

Distribution des Jobs (dispatching)

Une fois que vous avez écrit votre classe de job, vous pouvez la distribuer en utilisant la méthode **dispatch** sur le job lui-même. Les arguments passés à la méthode **dispatch** seront donnés au constructeur du job :

```
public function store(Request $request)
{
    $podcast = Podcast::create(/* ... */);

    // ...

    ProcessPodcast::dispatch($podcast);
}
```

Distribution retardé

Si vous souhaitez préciser qu'un job ne doit pas être immédiatement disponible pour être traité par un travailleur de la file d'attente, vous pouvez utiliser la méthode **delay** lors de la répartition du travail.

Par exemple, précisons qu'un job ne doit être disponible pour le traitement que 10 minutes après avoir été envoyé :

```
ProcessPodcast::dispatch($podcast)  
    ->delay(now()->addMinutes(10));
```